ĐẶC TẢ KIẾN TRÚC NGHIÊN CỨU — HSI REAL-CONDITIONED RESIDUAL GAN

Bản 0.9 | Để phản biện trước khi viết runner và sử dụng 30 giờ GPU

Bản tư vấn kỹ thuật và nghiên cứu

24/10/2026

TÓM TẮT QUYẾT ĐỊNH

Trạng thái: PROPOSED — REVIEW REQUIRED. Chưa có huấn luyện mới, chưa xác nhận hiệu quả hay tính mới thuật toán.

Hướng ưu tiên: thiết kế lại một GAN sinh **patch HSI thiểu số có điều kiện theo mẫu thật** (*Shared Real-Conditioned Residual Patch GAN*), kế thừa bài toán minority oversampling và trò chơi đối kháng của 3D-HyperGAMO. Indian Pines là dataset chính; V2 là mốc lịch sử, **không phải giới hạn kiến trúc**.

Giả thuyết H1 (khoa học): Khi có ít nhãn, Generator dùng chung tham số và nhận trực tiếp patch thật có thể học một phân bố biến thiên hữu ích quanh mẫu thiểu số, qua đó giúp một Classifier **cố định về kiến trúc** nhận diện lớp thiểu số tốt hơn so với lặp mẫu thật và augmentation đơn giản.

Giả thuyết H2 (một cải tiến có thể cô lập): Biểu diễn residual dạng **hạng thấp phổ–không gian** có thể tạo biến thiên hữu ích và kiểm soát được hơn đầu ra residual tự do có cùng vùng dịch chuyển, dưới cùng công thức loss, D, C và ngân sách.

- **Kiến trúc đề xuất cho thăm dò:** E_a + G_shared(x_anchor, z, class) -> residual -> x_fake; D(x, class); C(x).
- **Scaffold A0:** real-conditioned GAN với residual đầu ra tự do, bị giới hạn biên độ mềm và có kiểm soát *non-identity/noise dependence* bằng loss chung cho cả hai arm.
- **Candidate A1: cùng** GAN và **cùng loss**, chỉ đổi đầu residual sang factorization phổ–không gian hạng thấp. Đây là thay đổi kỹ thuật chính để kiểm tra; không thêm Transformer, metric learning, teacher hay đổi loss giữa hai arm.
- **Kết quả cần có sau ≤30 giờ GPU:** pipeline G/D/C lưu–tiếp tục được; A0/A1 đã được kiểm tra kỹ thuật; có đối chứng development với Real-only/ROS/augmentation; đã quan sát identity collapse, noise response, fidelity và chất lượng downstream; có quyết định GO / REVISE / STOP. **Không hứa có kết quả công bố hoặc một kiến trúc tối ưu.**

Điều kiện trọng yếu: Không coi validation cũ 100% hoặc test Protocol B đã được sử dụng là nơi chọn kiến trúc mới. Trước khi chạy dài, phải đo được khả năng đánh giá của ít nhất một development panel có sức phân biệt. Nếu chưa có panel ấy,

phần GPU chỉ được cấp cho kiểm tra kỹ thuật/profiling, không được gọi là tối ưu hiệu quả phân loại.

1. BÀI TOÁN, PHẠM VI VÀ LÝ DO CHỌN HƯỚNG

1.1. Bài toán ứng dụng

HSI gồm nhiều kênh phổ và cấu trúc không gian quanh pixel cần phân loại. Tập train thường mất cân bằng: có lớp rất đông và có lớp chỉ vài mẫu nhãn. Classifier dễ có kết quả OA tốt nhưng bỏ sót một số lớp ít mẫu. Mục tiêu chính là **cải thiện Macro-F1 và recall/F1 của các lớp thiểu số trên dữ liệu chưa dùng chọn mô hình**, không đơn thuần đẩy OA lên mức cao nhất.

1.2. Điều bắt buộc giữ và điều được phép thay

Bắt buộc giữ	Có thể thiết kế lại
Indian Pines làm dataset chính	Patch, PCA, biểu diễn và protocol của các run mới (phải ghi khác biệt)
GAN phục vụ minority oversampling, kế thừa 3D-HyperGAMO/GAMO	Generator G, Discriminator D, Classifier C, loss và optimizer
Chứng minh lợi ích ứng dụng cho mất cân bằng	Cách tạo mẫu, class sampling, schedule và chiến lược huấn luyện
Giữ hồ sơ V2/V3-A/M4 nguyên vẹn	Dataset bổ sung trong giai đoạn xác nhận sau

Khái niệm "kế thừa 3D-HyperGAMO" ở đây là **kế thừa bài toán G sinh mẫu thiểu số, D đối kháng, C phân loại**, không khẳng định còn đúng cơ chế *convex generator* của paper gốc. Phải ghi rõ đây là **redesign có nguồn gốc từ HyperGAMO**, không phải bản reproduction nguyên trạng. Nếu đổi nguyên tắc sinh lẫn loss, không được viện kết quả paper gốc như baseline chạy cùng điều kiện.

1.3. Bằng chứng nhóm đã có — và giới hạn

Theo báo cáo M4 được nhóm cung cấp, Indian Pines Protocol B 50e988fae260, seed 42/43/44:

Phương pháp	Macro-F1 (%), mean ± SD
Original GAN (tái triển khai của nhóm)	81,663 ± 2,877
Matched ROS	81,485 ± 0,967
Random-local	82,226 ± 3,004
Learned-local V2	83,009 ± 3,015

- V2 trộn 0.8 anchor + 0.2 weighted(3 same-class kNN); logits sinh toàn support rồi gather ba giá trị.

- Endpoint Softmax gần one-hot và ít đổi winner theo noise, **nhưng chưa chứng minh đây là nguyên nhân hiệu quả thấp**.
- V3-A thêm distance bias giảm Macro-F1 trên ba seed; không suy ra mọi điều kiện hóa theo khoảng cách đều thất bại.
- Endpoint ablation Learned vs Random-one-fixed là thí nghiệm **khác** M4; không trộn số liệu.
- Validation development gần/tròn 100%; test B từng được dùng trong quá trình phát triển; các patch cùng cảnh chồng lấn dấu chân nguồn. Không có phép xác nhận mới độc lập từ các kết quả ấy.

Kết luận từ bằng chứng: Nhóm có cơ sở nói cơ chế sinh mẫu cũ khó chứng minh lợi ích ổn định trong setup hiện có, **chưa có cơ sở nói residual GAN sẽ tốt hơn**. Mở một họ Generator mới là lựa chọn nghiên cứu có lý do, không phải kết quả đã được chứng minh.

1.4. Vì sao patch-conditioned GAN thay vì feature GAN hay Transformer ngay?

- **Patch-conditioned residual G:** có thể tạo patch 11×11×40 đưa trực tiếp cho nhiều Classifier, bảo toàn đầu vào nghiên cứu gốc; đổi lại khó bảo đảm ngữ nghĩa nhãn, D dễ overfit, G dễ sao chép anchor.
- **Feature GAN:** có thể nhẹ hơn, nhưng lợi ích lệ thuộc encoder E và không chứng minh sinh patch thật. Chỉ giữ làm **đường dự phòng thiết kế**, chưa train song song.
- **Transformer trong C:** có thể mạnh nhưng đồng thời thay C sẽ che hiệu ứng G. Chưa chọn làm C trong phép thử sinh mẫu đầu tiên. Khi phương pháp có ích với C cố định, mới kiểm băng CNN–Transformer trên cùng exposure.

Quyết định: ưu tiên **real-conditioned patch GAN**; không tuyên bố là họ kiến trúc mới trong văn liệu. **Cảnh báo nền tảng:** G(a,z,c)=a tạo đúng phân phối train của chính lớp đó khi anchor được lấy từ train. Một conditional GAN thuần adversarial vì thế có thể đạt cân bằng *bằng cách sao chép anchor*, không tạo dữ liệu mới! Đây không chỉ là lỗi kỹ thuật có thể xảy ra mà là **nghiệm suy biến hợp lệ của objective**. Do đó thiết kế phải chẩn đoán và có **ràng buộc non-identity/noise response** từ đầu; không thể chỉ đặt tên Residual GAN rồi tin G tự học biến thiên.

2. THAM CHIẾU HỌC THUẬT VÀ RANH GIỚI TÍNH MỚI

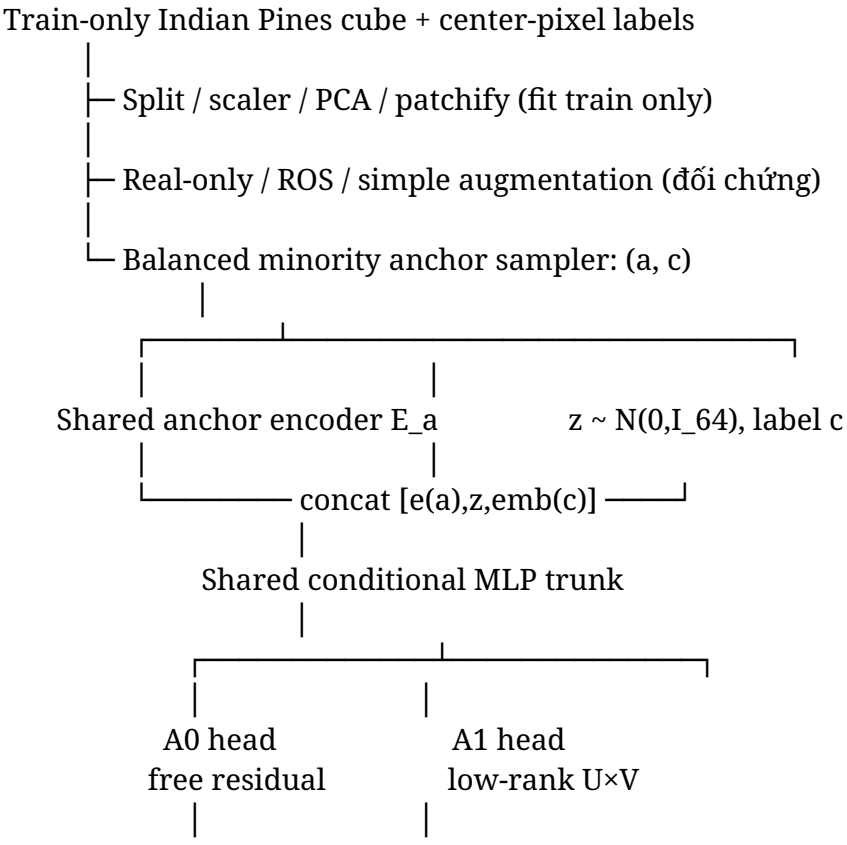
Nghiên cứu gốc	Kế thừa được	Không thể tự nhận là mới
GAMO, ICCV 2019 [R1]	G–D–C, sinh mẫu hữu ích cho ranh giới lớp ít dữ liệu	GAN oversampling hoặc sinh hard samples
3D-HyperGAMO, TGRS 2021 [R2]	Bài toán minority HSI patch và adversarial oversampling	Đưa GAN vào HSI
KPCA–SMOTE-guided GAN, JSTARS 2023 [R3]	Dẫn hướng G bằng thông tin thật có cấu trúc	Real/pattern-guided G
BAGAN-GP, 2021 [R4]	Học/chia sẻ cấu trúc giữa các lớp, D với dữ liệu mất cân bằng	Class-shared GAN cho imbalance
Limited-data GAN/ADA,	D có thể overfit khi có ít ảnh	D augmentation là thuật

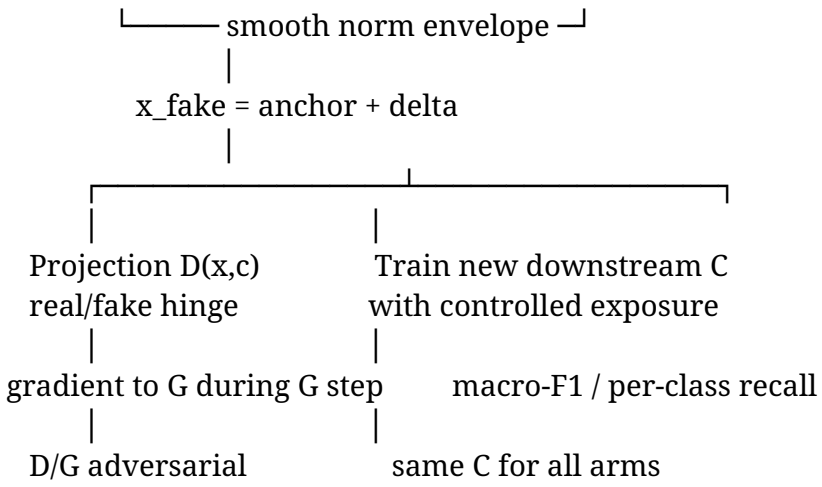
Nghiên cứu gốc	Kế thừa được	Không thể tự nhận là mới
NeurIPS 2020 [R5]		toán mới; áp dụng RGB augmentation bừa lên HSI
ADC-GAN, ICML 2022 [R6]	Class consistency có thể ảnh hưởng diversity; điều kiện lớp phải được xét kỹ	CE đúng nhãn = bảo đảm đúng semantics
S2A-Swin, Remote Sensing 2026 [R7]	Ràng buộc cấu trúc phổ–không gian, conditional GAN	GAN có ràng buộc phổ–không gian
3D-GIGAMO, Remote Sensing 2026 [R8]	Guided implicit generation, class-conditioned D, hybrid C	Generator dẫn hướng bởi patch thật hoặc kết hợp CNN–Transformer
HSI leakage survey [R9]	Kiểm tra sự phụ thuộc không gian khi chia patch	Random split có thể được coi là kiểm định không chồng lấn

Vị trí đóng góp có thể bảo vệ: một thiết kế biến thiên residual HSI *cụ thể* + thí nghiệm cô lập chứng minh ích lợi trong mất cân bằng. **Cho tới khi so đầy đủ prior art và có kết quả ablation, chỉ gọi là ứng viên cải tiến, không tuyên bố thuật toán mới.**

---

3. SƠ ĐỒ HỆ THỐNG ĐÍCH





Chú ý về C: Trong scaffold đầu tiên, C là **mô hình đánh giá downstream được huấn luyện tách biệt**, không phải teacher đồng tối ưu G. Đây là một quyết định có chủ đích để tránh GC adversarial target của M4 và tránh áp CE class-consistency lên G khi classifier ít mẫu chưa đáng tin. C vẫn là thành phần quan trọng của hệ *generation–classification*, nhưng **không nhận gradient về G** trong bản đầu. Nếu hội đồng yêu cầu giữ đúng trò chơi ba bên, có thể thêm nhánh *classifier-guided G* ở bước sau thành ablation riêng; không được nói bản scaffold đầu còn nguyên trò chơi GAMO ba bên.

---

4. DỮ LIỆU VÀ HỢP ĐỒNG TIỀN XỬ LÝ

4.1. Hợp đồng đầu vào chuẩn thăm dò

- Dataset: Indian Pines, ảnh gốc khoảng 145×145×200 và 16 lớp có nhãn. Không đào tạo trên nhãn pixel test.
- **Trong bản đầu:** giữ PCA 40 và patch 11×11 để so với hồ sơ M4 dễ hơn; đây là **default kỹ thuật**, được phép thay ở nghiên cứu cuối nếu có phép đối chứng tương ứng.
- Tensor vào mỗi mạng: X : float32 [B,11,11,40] (channel-last), nhãn y : int [B] (0..15).
- PCA/scaler fit chỉ trên các pixel/source được cho phép bởi train của split hiện hành. Nếu sử dụng một PCA đã fit train của Protocol B, chỉ dùng đúng phạm vi B; khi đổi split **phải fit lại**, không chuyển PCA chứa thông tin fold khác.
- Patch center chứa nhãn target; toàn patch có thể lẫn vật liệu/lớp khác, nên giữ nhãn trung tâm **không suy ra toàn bộ patch cùng lớp**.
- Dùng tọa độ tâm, class ID, split ID, transform hash và source graph trong metadata của mỗi patch; không dựa vào file order làm định danh.
- Các mẫu giả chỉ được sinh từ **anchor thuộc train của fold**; không dùng test hoặc dev validation làm donor, teacher pretraining hay tính thống kê lớp.

4.2. Class sampler

Trong G/D, chỉ oversample **15 lớp thiểu số được xác định từ train của fold**, không phát G cho majority trừ khi định nghĩa mục tiêu thay đổi.

sample c ~ Uniform(minority_class_ids)
sample anchor a ~ Uniform(train_supports_of_class_c)
sample z ~ N(0, I_64)

D lấy real x cùng nhãn c từ support thật của chính lớp đó; fake là G(a,z,c). D không được nhìn anchor cho fake mà không có anchor tương ứng cho real, để tránh lộ nhãn thật/giả theo cấu trúc input. C downstream được huấn luyện trên **16 lớp**, batch schedule của các arm phải công bằng.

Rủi ro: chỉ có 10 support ở Oats theo Protocol B; D theo lớp có thể ghi nhớ hoàn toàn. Phải đo gap điểm real-train vs held-in-train subset khác (nếu số lượng cho phép), duplicate-rate và entropy/variance D theo lớp; không coi GAN accuracy hay discriminator accuracy cao là mục tiêu.

---

5. G — GENERATOR CÓ ĐIỀU KIỆN THEO PATCH THẬT

5.1. Công thức tổng quát

Với anchor a và class c:

$$\tilde{x} = G_\theta(a, z, c) = a + \Delta_\theta(a, z, c), z \sim N(0, I_{64}).$$

- a, x_fake ∈ R^(11×11×40).
- Một bộ tham số G **dùng chung cho mọi lớp**; class conditioning đến từ embedding và class-balanced sampling.
- Noise z phải ảnh hưởng output theo thử nghiệm can thiệp (giữ a,c, đổi z). Có z trong input **chưa chứng minh** được dependence.

5.2. Encoder anchor E_a (kiến trúc đề xuất)

Tầng	Input	Output	Ghi chú
Conv2D(32,3×3, same), SiLU	[B,11,11,40]	[B,11,11,32]	Trộn PCA channels + spatial 3×3
Conv2D(64,3×3,same), SiLU	[B,11,11,32]	[B,11,11,64]	Không phải CNN 3D trên trục PCA
Conv2D(64,3×3,same), SiLU	[B,11,11,64]	[B,11,11,64]	GN/LN nếu cần, không dùng BatchNorm state chia giữa routes

Tầng	Input	Output	Ghi chú
GlobalAvgPool2D	[B,11,11,64]	[B,64]	Thông tin không gian gộp
Pixel-center vector	[B,11,11,40]	[B,40]	Giữ phổ pixel cần gán nhãn
Dense(64), SiLU	[B,40]	[B,64]	Nhánh center
Concat → Dense(128), SiLU	[B,128]	[B,128]	e(a)

Ước lượng nhanh: Encoder khoảng 0,09M tham số, tùy normalization và bias. Phải xác nhận bằng model.summary(), không dùng con số này làm kết quả thực nghiệm.

Lý do dùng Conv2D trộn 40 PC như channels: PCA components **không giữ thứ tự bước sóng vật lý**. Dùng convolution dọc trục PC rồi gọi nó là band-local spectral smoothness sẽ không có cơ sở vật lý tương ứng. Đây là lựa chọn phù hợp với dữ liệu PCA, **không khẳng định là encoder HSI mạnh nhất**.

5.3. Conditional MLP trunk

- e(a) ∈ R^128; z ∈ R^64; Embedding(c) ∈ R^16.
- Concat [B,208] → Dense(256)+LayerNorm+SiLU → Dense(256)+LayerNorm+SiLU → h ∈ R^256.
- Không dùng attention/Transformer trong G ở bản scaffold để giảm rủi ro kỹ thuật khi số support minority thấp.
- Các head được khởi tạo gần 0 (không **bằng 0 ở cả hai phía** nếu làm gradient biến mất), làm mẫu mới bắt đầu ở gần anchor rồi cho GAN học biên độ.

5.4. A0 — Scaffold: residual tự do có giới hạn mềm

$$T_{A0} = reshape\left(W_{free}h + b, 11, 11, 40\right).$$

Đầu ra W_free có 4.840 giá trị / sample. A0 dùng **cùng** envelope chuẩn hóa như A1; khác biệt cần so chỉ là **parameterization của direction**.

5.5. A1 — Candidate chính: factorized spectral–spatial residual

Chọn rank ban đầu R=4 (giả định kiến trúc, chưa phải k tối ưu):

$$U_\theta \in R^{11\times 11\times 4}, V_\theta \in R^{4\times 40},$$

$$T_{A1}[i,j,b] = \frac{1}{\sqrt{4}} \sum_{r=1}^{4} U_\theta[i,j,r] V_\theta[r,b].$$

Các tensor được sinh bởi hai Dense heads từ h: 484 đầu ra cho U, 160 cho V; phép einsum tạo [B,11,11,40].

- **Cơ sở:** residual trong mỗi sample có dạng ma trận 121×40 với rank tối đa 4; chia sẻ các hướng biến thiên phổ trên toàn patch, tránh cho mỗi pixel–band một tham số đầu ra độc lập.
- **Giới hạn:** hạng thấp **không bảo đảm** giữ phổ vật lý, nhãn trung tâm, độ đa dạng hoặc chất lượng classifier.
- **Điểm cần cô lập:** A0 và A1 giữ nguyên anchor, conditioning, D, loss và envelope; phép thử nhận định *structured residual có mang lại lợi ích không*. Tham số của hai head không bằng nhau: nếu mục tiêu là hiệu quả tham số, ghi riêng; nếu muốn kiểm định cấu trúc độc lập khỏi số tham số, cần thêm đối chứng A0 có width match ở giai đoạn sau.

5.6. Soft norm envelope — tránh hard clipping triệt gradient theo amplitude

Đặt s_a là trung vị khoảng cách Frobenius từ anchor đến tối đa 3 support khác cùng lớp gần nhất **trong train**. Với lớp chỉ vài support, ghi số neighbor thực dùng. Đặt:

$$R_a = \eta\, s_a, \eta = 0.25 \backslash(\text{giá trị khởi tạo cần thẩm định bằng train-only stats}\backslash).$$

Cho T là output A0 hoặc A1, dùng:

$$\Delta = \frac{R_a T}{\sqrt{R_a^2 + \| T \|_F^2 + \epsilon}}, \tilde{x} = a + \Delta.$$

Hệ quả đúng về đại số:

$$\| \Delta \|_F < R_a \left(R_a > 0 \right).$$

Ở miền ||T|| ≪ R_a, Delta ≈ T. Khi ||T|| rất lớn, norm tiệm cận R_a, không bị *hard-clipping* với đạo hàm amplitude bằng 0 ngay tại vùng ngoài biên. **Gradient vẫn có thể rất nhỏ khi T lớn**, nên phải đo gradient thực.

Khác trust-region cũ, R_a dựa trên cùng lớp và **không mang bảo đảm gần anchor hơn mọi lớp khác**, càng không bảo đảm đúng nhãn. Biên độ này là **giả thuyết kiểm soát**, chưa được chứng minh thích hợp. Phải báo tỷ lệ ||Delta||/R_a và khoảng cách center/full patch. Nếu s_a gần 0 vì support trùng lặp, mẫu dễ co về anchor; cần kiểm tra phân bố s_a trên CPU **trước** full training và có ngưỡng STOP nếu đa số gần zero.

5.7. Identity equilibrium là vấn đề toán học bắt buộc xử lý

Giả sử anchor được lấy đúng từ phân phối mẫu thật theo lớp: a ~ p_train(x|c). Nếu G(a,z,c)=a với mọi z, thì phân phối đầu ra của G **bằng phân phối train thật**; D thuần real/fake không có cách phát hiện rằng Generator chỉ đang sao chép. Nói khác đi:

$$p_G(x \mid c) = p_{train}(x \mid c) \text{ ngay cả khi } G(a, z, c) = a.$$

Đây là **nghiệm suy biến của trò chơi**, không chỉ là một rủi ro kiến trúc. Không có giả thiết nào cho phép kết luận adversarial loss tự nó sẽ buộc G tạo biến thiên hữu ích. Các công trình guided GAN cũng phải xử lý vấn đề tương tự tùy mục tiêu.

Vì thế A0 **và** A1 phải dùng cùng một phép kiểm soát yếu *non-identity + noise response*, và phải báo rõ rủi ro đánh đổi:

$$L_{nov} = E\left[\left(\frac{max\left(0, \tau_{nov} s_a - \| G(a,z,c) - a \|_F\right)}{s_a + \epsilon}\right)^2\right],$$

$$L_{noise} = E\left[\left(\frac{max\left(0, \tau_{noise} s_a - \| G(a,z_1,c) - G(a,z_2,c) \|_F\right)}{s_a + \epsilon}\right)^2\right].$$

Đề xuất khởi tạo: tau_nov=0.03, tau_noise=0.02 (tỷ lệ của khoảng cách cùng lớp train-only); đây chỉ là **điểm bắt đầu cần kiểm soát gradient**, không phải ngưỡng đã tối ưu. Giới hạn trên R_a≈0.25 s_a tạo khoảng khả thi về độ lớn nếu hướng sinh và D hỗ trợ. Nếu s_a=0, loại khỏi phép tỷ lệ, báo tỷ lệ mẫu và xử lý rõ ràng; không chia cho số rất nhỏ rồi gọi đa dạng.

Mục tiêu G của scaffold sẽ là:

$$L_G = L_G^{adv} + \lambda_{nov} L_{nov} + \lambda_{noise} L_{noise}.$$

lambda_nov và lambda_noise phải được **khóa giống nhau cho A0/A1** sau kiểm tra ngắn trên train-only: chọn trọng số sao cho gradient regularizer không áp đảo adversarial gradient, ghi log gradient norm từng thành phần, không dùng test/held-out F1 để dò λ. Có thể khởi đầu 0.05/0.05 cho smoke, **chưa phê duyệt làm giá trị full training**. Mỗi batch tính hai noise cho cùng anchor để đo L_noise, chi phí thật phải đưa vào profiler.

Cảnh báo: ép G khác anchor hoặc nhạy noise **không chứng minh mẫu đúng lớp**. Generator có thể đáp ứng penalty bằng nhiễu vô nghĩa hoặc chuyển sang support thật khác cùng lớp. D và downstream utility là đối chứng bắt buộc; nếu chỉ thấy L_noise giảm nhưng F1 không tăng, không được gọi thành công. Dùng cùng penalty cho cả hai arm giúp so sánh A0/A1 vẫn cô lập được **parameterization residual**.

5.8. Vì sao không thêm class-preserving loss hoặc residual spectral smoothness ngay?

- Cross-entropy từ một C train trên vài support có thể khuyến khích sao chép anchor hoặc sinh mẫu dễ, giảm diversity; ADC-GAN [R6] cho thấy có vấn đề về intra-class diversity trong auxiliary-classifier GAN.
- Spectral smoothness trực tiếp qua chỉ số 40 PC là sai về thứ tự vật lý. Nếu cần, xử lý trong 200 raw bands hoặc PCA inverse transform với scaler nhất quán, và đánh giá độ mất thông tin từ PCA.
- Spatial TV có thể làm phẳng biên vật liệu thật trong patch hỗn hợp. Chỉ dùng sau khi chứng minh nhiễu high-frequency là failure mode.
- Không dùng hard maximum distance đến lớp khác: có thể bóp không gian sinh và gây triệt gradient của mixing-strength.

Quyết định bản đầu: soft norm envelope và hai penalty chống suy biến là **các thành phần chung của scaffold**. **Một cải tiến chính duy nhất cần đánh giá** là đổi T_A0 sang T_A1. Không thêm teacher, hardness loss hoặc Transformer trong lượt này. Đây vẫn chỉ là giả thuyết

cần smoke và thử nghiệm downstream; nếu non-identity penalty làm sai nhãn hoặc training bất ổn, dừng thay vì thêm module tùy ý.

---

6. D — CLASS-CONDITIONAL DISCRIMINATOR

6.1. Thiết kế đề xuất

Đầu vào x ∈ [B,11,11,40], nhãn class c:

Khối	Output	Ghi chú
Conv2D 32,3×3, stride 1 + LeakyReLU	[B,11,11,32]	SN ưu tiên nếu framework hỗ trợ ổn định
Conv2D 64,3×3,stride 2 + LeakyReLU	[B,6,6,64]	Không dùng BatchNorm
Conv2D 128,3×3,stride 2 + LeakyReLU	[B,3,3,128]	Không dùng BatchNorm
GlobalAvgPool + Dense 128	[B,128]	Feature h_D(x)
Projection score	[B,1]	D(x,c)=w^T h_D(x) +<h_D(x),e_c>/sqrt(128)

D được condition theo lớp, nhưng **không nhìn anchor**. Class sampling của real và fake phải cùng chính sách; nếu D nhìn class frequency khác giữa real/fake, nó học shortcut thay vì tính chân thực patch.

6.2. Loss adversarial được chọn làm scaffold: conditional hinge

$$L_D = E_{(x,c)\sim p_{real}^{bal}}\left[max\left(0, 1 - D(x,c)\right)\right] + E_{a,z,c}\left[max\left(0, 1 + D\left(G(a,z,c),c\right)\right)\right],$$

$$L_G^{adv} = -E_{a,z,c}\left[D\left(G(a,z,c),c\right)\right].$$

- D output **score không giới hạn**, không phải xác suất; không dùng sigmoid hoặc nhãn 1−condition kiểu route GC M4.
- Ưu tiên spectral normalization (nếu smoke trong phiên bản TF/Keras thực sử dụng hoạt động đúng); **không đồng thời chọn WGAN-GP và hinge**.
- stop_gradient(G(a,z,c)) khi cập nhật D; D weights giữ cố định khi cập nhật G, nhưng gradient qua forward D vẫn phải truyền đến G.
- Adam(G,D) độc lập: lr=2e-4, beta1=0.0, beta2=0.9 là **mốc kỹ thuật đề xuất** từ thông lệ hinge GAN, không phải thông số đã tối ưu cho Indian Pines; cùng một cấu hình phải được khóa cho A0/A1.

Chưa chọn adaptive discriminator augmentation, R1 hoặc gradient penalty trong bản đầu. Chỉ xem xét một biện pháp nếu profiler/chẩn đoán cho thấy D overfit hoặc huấn luyện bất ổn. HSI

không áp RGB color jitter hay các phép dịch không gian làm đổi ngữ nghĩa pixel trung tâm một cách mặc định.

---

7. C — CLASSIFIER DÙNG ĐỂ CHỨNG MINH GIÁ TRỊ OVERSAMPLING

7.1. Vai trò và tách hiệu ứng

C trong bản scaffold là classifier được train **từ đầu ở mỗi arm** trên cùng data protocol và budget. Nó không đồng tối ưu với G trong GAN scaffold. Chúng ta đo hai câu hỏi tách biệt:

1. Hệ G + D + C mới hoạt động/cho điểm thế nào?
2. **Riêng mẫu do G sinh** đem lại bao nhiêu lợi ích so với ROS và augmentation với cùng C?

Việc C không cấp GC gradient là **khác biệt có chủ ý so với GAMO ba người chơi gốc**. Nếu muốn đưa C vào objective của G sau này thì đó là một *phần mở rộng có đối chứng* và không được so trực tiếp hai recipe rồi gọi mọi khác biệt là do Generator architecture.

7.2. Cố định một CNN spectral–spatial đủ năng lực

Đề xuất một C giữ nguyên qua tất cả arm:

input [B,11,11,40]
Conv2D(64,3,same) + LayerNorm + GELU
Conv2D(64,3,same) + GELU + residual shortcut
Conv2D(96,3,same) + LayerNorm + GELU
GlobalAvgPool -> [B,96]
center spectrum 40 -> Dense(64) + GELU -> [B,64]
Concat [B,160] -> Dense(128) + GELU + Dropout(0.15)
Dense(16) logits -> Softmax only for probabilities

- Loss classifier: sparse categorical cross-entropy, sampling được khóa; AdamW lr≈3e-4 là **khởi tạo kỹ thuật**, không mặc nhiên tốt nhất.
- **Một lựa chọn imbalance duy nhất cho Real-only:** class-balanced sampling hoặc loss class-balanced (phải khóa trước); không kết hợp các biện pháp khác nhau theo arm.
- Giai đoạn thăm dò đầu **không train SSFTT/Transformer**. Chỉ khi A1 cải thiện G-specific trên C cố định mới dùng cùng một CNN–Transformer để kiểm khả năng chuyển giao; khác framework PyTorch/TF làm tăng chi phí triển khai.

7.3. Công bằng về classifier exposure

Mỗi arm dùng **cùng số lần C update, cùng batch size, cùng phân phối class ở batch, cùng real/fake slots**. Khi arm không có fake, lấp fake slot bằng real reuse cho matched ROS; Real-only báo riêng như baseline huấn luyện cùng ngân sách. Khi một arm có fake, trong cùng class slot được chọn mẫu từ G; schedule này được định nghĩa trước, không biến theo kết quả test.

- C init/seed giống nhau theo *paired arm* khi có thể, nhưng RNG trajectories không tuyệt đối giống khi khác dữ liệu.
- Không chuyển số epoch giống nhau thành exposure giống nhau khi số lượng mẫu tổng hợp khác nhau.
- Generator được freeze khi train downstream C để **so chất lượng dữ liệu sinh**, tránh C cùng G thay đổi qua từng arm.
- Nếu sinh offline, khóa num_fake_per_class, fake seed và anchor selection; báo bằng số lượng real/fake thực tế đã xem trong train.

---

8. TRAINING PROTOCOL — CÁC PHA VÀ ĐƯỜNG GRADIENT

8.1. Pha -1: dữ liệu và development protocol (CPU, trước GPU dài)

1. Sinh manifest theo split_id, class_id, coordinate, patch_shape, preprocessing_fit_scope, pca_hash.
2. Kiểm tra số lượng của 16 lớp và tỷ lệ trùng footprint giữa các nhóm; không tự ý bỏ Oats hoặc đổi patch mà không đổi tên benchmark.
3. Tạo **all-16 development panel** từ train được phép, đo xem nó có trần accuracy hay thiếu lớp. Nếu mọi C đạt ~100%, panel không đủ xếp hạng chất lượng; chỉ dùng để smoke/monitor.
4. Đánh giá **spatial stress panel** riêng nếu số lớp/mẫu còn đủ. Panel thiếu lớp **không** được báo Macro-F1 all-16. Split mới trên cùng Indian Pines vẫn không phải independent-scene validation.
5. Nếu cả hai panel không phân biệt được và không có dữ liệu xác nhận sạch, **không dùng GPU để tối ưu architecture theo Macro-F1**; chỉ được phép phát triển scaffold kỹ thuật.

8.2. Pha 0: kiểm thử mô hình, chưa full train

- Unit tests shapes, finite, output norms và biên $||Delta|| < R_a$; kiểm noise non-constant ở initialization.
- D projection dùng class thật trong real/fake cùng distribution.
- Kiểm GradientTape: D train chỉ cập nhật D; G train gradient D→fake→G không None; C train không cập nhật G/D.
- Không gọi smoke **PASS** là bằng chứng downstream F1 sẽ tăng.
- Profile wall time và VRAM thật sau warmup (có đồng bộ GPU khi đo), kể cả preprocessing/generate/classifier/checkpoint.

8.3. Pha 1: training GAN scaffold

Một bước GAN đề xuất:

```
# c balanced across 15 minority classes
c, a_real = sample_balanced_anchor()
x_real = sample_real_same_class(c)
```

```
z = normal([batch, 64])

# Update D once
x_fake = stop_gradient(G(a_real, z, c))
loss_D = mean(relu(1 - D(x_real, c))) + mean(relu(1 + D(x_fake, c)))
apply_gradients_D(loss_D)

# Update G once (new z or predefined paired RNG)
z2 = normal([batch, 64])
x_fake = G(a_real, z2, c)
loss_adv = -mean(D(x_fake, c))
# Paired noise probe on the SAME anchors (additional G forward)
z3 = normal([batch, 64])
x_fake_other = G(a_real, z3, c)
loss_nov = normalized_lower_bound_displacement(x_fake, a_real, s_anchor)
loss_noise = normalized_pair_distance_floor(x_fake, x_fake_other, s_anchor)
loss_G = loss_adv + lam_nov * loss_nov + lam_noise * loss_noise
apply_gradients_G(loss_G)
```

- Bắt đầu **1D : 1G update**, không copy 34 slots/loop của M4; schedule mới có lý do vì GAN architecture/loss mới. Nhưng mọi arm A0/A1 phải dùng cùng schedule.
- D phải được train trên real/fake từ cùng tập minority classes; nếu đổi class sampling, đó là intervention khác.
- Checkpoint đầy đủ (E_a,G,D, optimizers, RNG, c_sampler_state, step, PCA/split/config hash); tiếp tục train đúng config từ checkpoint, **không chỉnh optimizer giữa pilot và full rồi gọi resume đúng cùng recipe**.
- Khi D quá mạnh ở lớp hiếm, không lập tức mở đồng thời ADA + WGAN-GP + class-consistency; chỉ chọn **một** thay đổi sau báo cáo failure.

8.4. Pha 2: huấn luyện C từ các augmentation arm

Dùng **cùng C, class schedule và budget**, chạy từ initialization được ghi lại:

- R: Real-only (baseline).
- ROS: same-class real repetition, matched exposure.
- SIMPLE: local/mixup augmentation đơn giản có kiểm soát; pre-register công thức và mức trộn, không lấy V2 test để tune.
- A0: fake từ shared real-conditioned GAN với residual tự do.
- A1: fake từ chính GAN scaffold nhưng residual factorized rank 4.

Đối với A0 và A1, bắt buộc **mỗi arm có GAN được train riêng** vì khác output parameterization; việc thay head sau khi train A0 **không thay cho ablation huấn luyện từ đầu**. Với 30 giờ, nếu không đủ ngân sách hai full GAN, báo pilot và hoãn kết luận hiệu quả causal.

8.5. Pha 3: xác nhận sau 30 giờ (không thuộc cam kết thăm dò)

- Nhiều seed/split được khóa; giới hạn độ tin cậy trên cùng một cảnh; đánh giá ít nhất một dataset khác.
- Re-evaluate Original HyperGAMO/V2 dưới **cùng** preprocessing, C, split và training budget nếu muốn xếp hạng trực tiếp.
- Kiểm data augmentation với một C mạnh khác (CNN–Transformer) như phép thử khả năng chuyển giao, không dùng để quy điểm toàn hệ thống cho G.

9. VÌ SAO KIẾN TRÚC CÓ THỂ THẤT BẠI? CẦN ĐO GÌ?

Failure mode	Dấu hiệu phải đo	Diễn giải đúng / xử lý
G sao chép anchor (identity)	d_full=||fake-a||/s_a và median d_full→0	D có thể xem duplicate là real; **GAN không mang lại oversampling hữu ích**. Không tùy ý thêm noise penalty sau khi thấy test.
G bỏ qua z	Var/median pair-distance G(a,z1,c) vs G(a,z2,c)	Cần báo noise dependence theo anchor/lớp; conditioning không tự bảo đảm diversity.
D ghi nhớ rare supports	D gap train-real vs held-aside train-only supports, high train D score gap	Class-balanced batch không tạo thêm độc lập dữ liệu. Cân nhắc D regularization một yếu tố/1 lần.
Sample lệch nhãn	Frozen real-only C reference, center spectrum nearest-class, manual spectral diagnostics	Proxy labels không phải chứng minh ground truth; giảm amplitude/hủy candidate nếu chỉ tăng fake diversity.
Model biến đổi toàn context sai nhãn tâm	Distances center-vs-context; class confusion around boundary	Nhãn của patch do center quyết định; giữ center information, báo lỗi quanh boundaries.
Low-rank quá hạn chế	rank(T), singular values, sample covariance, F1	A1 có thể thua A0; không tự tăng rank cho tới khi có bằng chứng và ngân sách.
Soft envelope ép residual nhỏ	R_a, ||T||/R_a, ||Delta||/R_a per-class	Nếu gần mọi mẫu ở anchor, STOP; không coi giới hạn norm là label guarantee.
G sinh nhiều có sức đánh lừa D	Center C/proxy confidence, high-frequency spatial energy	Tăng diversity không đồng nghĩa tốt.

Failure mode	Dấu hiệu phải đo	Diễn giải đúng / xử lý
Chỉ một lớp nhỏ làm F1 tăng	Per-class F1, recall, confusion, group counts	Không tuyên bố lợi ích chung cho minority từ một lớp test cực nhỏ.
Validation không đủ phân biệt	Same C đạt gần 100% ở mọi arm	NO-GO cho claim improvement trên panel ấy, dù gradient train pass.

Chỉ số chẩn đoán bắt buộc: GAN loss D/G, D score real/fake theo lớp; identity ratio, noise effect, synthetic-to-anchor/synthetic-to-real neighbor distances, support usage, spectral/spatial residual energy, downstream Macro-F1 và per-class recall, wall time route + peak VRAM. Không đánh giá GAN chỉ bằng FID RGB mặc định; HSI PCA/ít support làm FID không ổn định và phụ thuộc feature extractor.

---

10. KẾ HOẠCH PHÂN BIỆT HIỆU ỨNG VÀ QUY TẮC KẾT LUẬN

10.1. Primary comparison

A1 vs A0 trên cùng development protocol, cùng D/C/loss/class sampler/budget; và cả hai so với ROS/SIMPLE trên cùng C.

- A1 > A0 trên nhiều điều kiện mà không chỉ tăng một lớp nhỏ → tín hiệu tốt cho structured residual.
- A1 ~ A0 theo point estimate **không có nghĩa tương đương**; độ bất định có thể quá lớn.
- A1/A0 không tốt hơn ROS/SIMPLE → chưa chứng minh GAN có lợi ích đáng trả chi phí.
- A1 > ROS mà A1 ≈ A0 → có tín hiệu real-conditioned GAN hữu ích, **chưa chứng minh low-rank là đóng góp**.

Hình thức báo cáo: paired difference theo seed, mean±SD, per-class F1/recall, hiệu ứng theo từng split/panel, runtime và số run; nếu làm bootstrap phải nêu đơn vị lấy mẫu và sự phụ thuộc không gian. n=3 seeds trên cùng scene không thành 3 scene độc lập.

10.2. Không dùng các cách diễn giải sai

- Không so OA 99,37% GIGAMO với OA 88,251% M4 rồi kết luận generator nào mạnh hơn.
- Không dùng test Protocol B để quyết định A1 hay A0.
- Không tuyên bố giảm tham số G là giảm thời gian tương ứng; đo wall clock route + end-to-end.
- Không gọi giảm F1 để đổi runtime là Pareto dominance.
- Không gọi predictor C_ref xác nhận nhãn vật lý của fake.
- Không gọi structured residual mới tuyệt đối khi literature đã có guided GAN, spectral–spatial constraints, low-rank priors ở nhiều miền.

10.3. Bảng quyết định tiền đăng ký

Quan sát	Quyết định hợp lệ
A1 có downstream gain nhất quán trên panel có sức phân biệt; ROS/SIMPLE thấp hơn; non-copying và noise-sensitive	**GO**: giữ kiến trúc A1 cho giai đoạn xác nhận nhiều seed/dataset.
A0 hữu ích hơn A1 nhưng A1 quá gò bó, identity cao	**REVISE**: xem lại rank/envelope; không tuyên bố H2.
A0/A1 đều giống ROS và identity cao, D dễ bị đánh lừa	**STOP** claim GAN utility; giữ các baseline, không train thêm vì kiến trúc đẹp.
Mọi arm đạt ~100% trên validation, không giữ đủ 16 lớp trên panel khác	**INCONCLUSIVE**: chưa xếp hạng được; ưu tiên giao thức/dataset, không chạy full search.
F1 tăng do một class chỉ vài test pixel, seed dao động lớn	**INCONCLUSIVE**: không chọn vì point estimate.
GPU budget hết mà chỉ có 1 GAN run	Báo **pilot kỹ thuật** và giới hạn, không suy ra superiority.

11. 30 GIỜ GPU — LẬP NGÂN SÁCH TỪ PROFILER, KHÔNG HỨA CHẮC

11.1. Nguyên tắc

30 giờ là **trần phát triển**, không phải số giờ phải tiêu hết. Nếu dùng hai GPU song song, phải phân biệt wall-hours và device-hours, cùng chính sách tính quota của nhà cung cấp. Thời gian người viết code, xem output và sửa lỗi không biến mất dù còn GPU quota.

Một lượt GPU được phê duyệt khi: có claim cụ thể; dữ liệu đánh giá còn sức phân biệt; có baseline đối chứng; code/mô hình đúng; đã đo thời gian thật và tổng kinh phí còn đủ làm comparison.

11.2. Dự toán đề xuất sau profiling (không phải xác nhận runtime)

Hạng mục	Trần tham khảo	Mục đích
Smoke, training-mode checks, profiler	2 device-hours	Xác minh và ước lượng T_GAN/T_C
Fixed C Real-only/ROS/SIMPLE	6 device-hours	Đo panel đánh giá + cơ sở augmentation
Một GAN scaffold A0 và downstream C	9 device-hours	Identity/noise/fidelity/utility
Một GAN candidate A1 và downstream C	9 device-hours	H2, same-recipe comparison
Buffer/export/inference/retry	4 device-hours	Hoàn thành kết quả có thể tái lập

Hạng mục	Trần tham khảo	Mục đích
Tổng trần	**30 device-hours**	**Chỉ hợp lệ nếu profiler xác nhận đủ**

Nếu T_GAN/T_C lớn hơn dự toán, **giảm số lượt / chuyển một arm thành pilot**, không tự ngầm cắt update của một arm để tạo vẻ công bằng. Thứ tự ưu tiên bảo vệ GPU: baseline có sức phân biệt → A0 → A1 → ablation mở rộng. Nếu không còn giờ cho A1, phải ghi H2 chưa kiểm định.

11.3. Benchmark tối thiểu trước full

- Warmup vài bước có compile, sau đó đo ít nhất một đoạn đủ ổn định có GPU synchronize.
- Thời gian riêng E/G-forward, D-step, G-step, G fake-generation, C-step, validation/inference, checkpoint write/read.
- Peak allocated/reserved GPU memory, lưu/sinh fake offline, I/O bottleneck.
- Ước tính (steps_G*sec_G + steps_D*sec_D + steps_C*sec_C + overhead) với **hệ số dự phòng**; cộng chi phí code/smoke/retry ngoài device-hours.
- Không cố áp 8.000 C updates/4.000 loops và shared legacy Adam của M4 cho GAN mới; schedule khác phải được trình bày như **redesign**.

12. HỢP ĐỒNG THỰC THI CHO CODEX

12.1. API tối thiểu

```
make_split_manifest(dataset, policy, seed) -> manifest
fit_preprocess(train_manifest) -> scaler, pca
patchify(source_cube, manifest, pca) -> patches, labels, coordinates
build_anchor_encoder(input_shape=(11,11,40)) -> Ea
build_generator(mode='A0'|'A1', z_dim=64, num_classes=16) -> G
build_discriminator(num_classes=16) -> D
build_classifier(num_classes=16) -> C
train_gan(G, D, train_dataset, config) -> checkpoints, logs
export_fake(G, train_dataset, exposure_spec, seed) -> fake_manifest
train_classifier(C, real_data, fake_manifest|None, fixed_budget) -> ckpt
report_eval(ckpts, panel, class_list) -> tables, plots
```

12.2. Bắt buộc log và lưu

run_id, git_hash, config_hash, dataset/split/PCA hash, seed_python/numpy/tf, optimizer_states, RNG state, sampler position, steps_G/D/C, elapsed GPU time, dtype, device, class counts, augmentation exposure, endpoint checkpoints, all-16/classwise metrics, norm/noise diagnostics, sampler integrity.

Checkpoint resume: load đúng G,D,E_a, optimizer iteration/moments, sampler/RNG; không chỉ load weights. Test resume bằng một đoạn ngắn và so hành vi với run liên tục, chỉ khẳng định deterministic nếu toàn bộ kernel và trạng thái cho phép.

Code quality gates: shape tests; finite loss/grad; no NaN; gradient flows đúng; proper train/eval mode; no validation/test donors; stochastic fake differences under z; output residual bound; class-label mapping stable; no silent PCA refit on validation/test.

12.3. Danh sách quyết định cần người triển khai xác nhận

Quyết định	Mặc định bản 0.9	Cần xác nhận trước full?
Dùng PCA40/P11 cho prototype	Có	Có — audit split/preprocess
G-input noise 64	Có	Khóa trong config
G shared across 15 minority	Có	Có — class sampling
E_a 3 Conv2D + center branch	Có	Kiểm model.summary()
A0 free vs A1 rank4	Một thay đổi chính	Có — chi phí đủ cả hai run?
Envelope scale η=.25	Mốc thử kỹ thuật, chưa tối ưu	**Có — train-only distance distribution**
D projection + spectral norm + hinge	Có	**Có — check TensorFlow implementation**
C CNN+center spectrum	Có	Có — real-only baseline và dev panel
C gradient vào G	**Không ở scaffold đầu**	Phải ghi rõ deviation khỏi GAMO
Chống identity/noise collapse	L_nov + L_noise giống nhau A0/A1	**Có — scale λ bằng gradient train-only, đo artifact**
Transformer, WGAN-GP, teacher, trust-region	Không	Không thêm vào full run đầu
Label-consistency loss	Không	Chỉ khi có bằng chứng proxy đáng tin

13. NHỮNG GÌ BẢN NÀY CHƯA CHỨNG MINH

1. **Chưa chứng minh hiệu quả:** Không có kết quả train GAN residual mới, chưa có đo noise dependence hay identity thực tế.
2. **Chưa audit source thật M4:** Phần source hiện chỉ được người dùng/báo cáo mô tả; đặc tả này không thay cho đọc m4.py, tf version và runner.
3. **Chưa có protocol validation dùng được:** Cần dữ liệu tọa độ và class counts để đo feasibility; không khẳng định sẽ tạo được all-16 source-disjoint panel trên Indian Pines.

4. **Chưa chứng minh spectral validity:** PCA40 là biểu diễn nén. Norm-bounded residual và low rank chỉ là prior hình học, không phải vật lý/nhãn đúng.
5. **Chưa xác lập novelty:** Guided patch GAN, conditional GAN, spectral–spatial priors đều có paper trước. Cơ chế cụ thể phải đối chiếu sâu thêm và có ablation thật trước khi đặt claim.
6. **Chưa biết đủ 30 giờ không:** Đòi benchmark trên GPU thật, nhiều lượt C có thể chiếm phần lớn ngân sách.
7. **Chưa chứng minh so với GIGAMO:** Những kết quả paper khác protocol chỉ là related work; so riêng G bằng cùng classifier, split và exposure.

14. QUYẾT ĐỊNH ĐỀ NGHỊ ĐỂ NGƯỜI DÙNG DUYỆT

Chọn hướng A để chuẩn bị triển khai scaffold có kiểm soát identity/noise; chọn *structured residual A1* là ứng viên cải tiến duy nhất của vòng 30 giờ, so với *free residual A0*.

- **GO để viết code và chạy tests kỹ thuật:** Sau khi có source, split, PCA và GPU runtime.
- **CHƯA GO full 30 giờ:** Cho tới khi dev protocol, train-only residual scale và profiler cho thấy có thể đánh giá so sánh có ích.
- **Tiêu chí dừng:** C chủ đạo không phân biệt được trên development; Generator copy anchor/bỏ noise dù có regularizer hoặc sinh nhiễu sai lớp; GAN không hơn augmentation; không đủ runtime cho A0/A1 cùng ngân sách.
- **Hướng cuối cùng chưa bị ràng buộc:** Nếu A1 có bằng chứng tốt, mới nghiên cứu C-guidance, spectral-prior raw-band, D low-data augmentation và dataset thứ hai. Không tự gộp các module đó vào lần train đầu.

Một câu kết luận: Xây GAN sinh patch thiểu số từ mẫu thật là hướng phù hợp với quyền thiết kế lại mà nhóm mong muốn. Nhưng giá trị khoa học của nó sẽ do **đối chứng cùng Classifier, ability to generalize và sample utility** quyết định — không phải việc G có thể tạo ra nhiều tensor mới.

TÀI LIỆU NGUỒN SƠ CẤP VÀ ĐỐI CHIẾU

- **[R1]** Mullick et al. (2019), *Generative Adversarial Minority Oversampling*, ICCV. https://openaccess.thecvf.com/content_ICCV_2019/html/Mullick_Generative_Adversarial_Minority_Oversampling_ICCV_2019_paper.html
- **[R2]** Roy et al. (2021), *Generative Adversarial Minority Oversampling for Spectral-Spatial Hyperspectral Image Classification*, IEEE TGRS. DOI 10.1109/TGRS.2021.3052048. Code https://github.com/mhaut/3D-HyperGAMO
- **[R3]** Miftahushudur, Grieve & Yin (2023), *Permuted KPCA and SMOTE to Guide GAN-Based Oversampling for Imbalanced HSI Classification*, IEEE JSTARS. https://doi.org/10.1109/JSTARS.2023.3326963
- **[R4]** Huang et al., *Enhanced balancing GAN: minority-class image generation* (2021). https://pmc.ncbi.nlm.nih.gov/articles/PMC8211314/

- **[R5]** Karras et al. (2020), *Training Generative Adversarial Networks with Limited Data*, NeurIPS. https://papers.neurips.cc/paper_files/paper/2020/hash/8d30aa96e72440759f74bd2306c1fa3d-Abstract.html
- **[R6]** Hou et al. (2022), *Conditional GANs with Auxiliary Discriminative Classifier*, ICML. https://proceedings.mlr.press/v162/hou22a.html
- **[R7]** *S2A-Swin: Spectral Smoothing–Guided Spectral–Spatial Windows with Generative Augmentation for HSI Classification Under Class Imbalance and Limited Labels*, Remote Sensing (2026). https://www.mdpi.com/2072-4292/18/6/935
- **[R8]** *A Guided Implicit Generative Adversarial CNN–Transformer Framework for Class-Imbalanced Hyperspectral Image Classification* (3D-GIGAMO), Remote Sensing (2026). https://www.mdpi.com/2072-4292/18/19/3343
- **[R9]** *Information Leakage in Deep Learning-Based Hyperspectral Image Classification: A Survey*, Remote Sensing (2023). https://www.mdpi.com/2072-4292/15/15/3793
- **[R10]** *Rebooting ACGAN: Auxiliary Classifier GANs with Stable Training*, NeurIPS (2021). https://proceedings.neurips.cc/paper_files/paper/2021/hash/c5ab6cebaca97f7171139e4d414ff5a6-Abstract.html

Nguồn nội bộ: bản tóm tắt kết quả V2/V3-A/M4 và tài liệu Thiết kế lại GAN — phạm vi và lựa chọn sơ bộ do nhóm cung cấp ngày 24/10/2026; toàn bộ số liệu mô hình cũ trong báo cáo trên được ghi rõ là do nhóm cung cấp, không phải kết quả độc lập mới.